DuocUC | ONLINE

DESARROLLO CLOUD NATIVE (CDY2204)

Semana 3 — Evaluación Sumativa

Construyendo y desplegando una solución Cloud Native

Sistema de Gestión de Reservas de Eventos con Tickets — Spring Boot + Amazon EFS + Amazon S3 + CI/CD (GitHub Actions, Docker Hub, EC2)

Campo	Detalle
Integrante(s)	[COMPLETAR]
Carrera	Analista Programador
Docente	[COMPLETAR]
Sección	[COMPLETAR]
Repositorio	[COMPLETAR: https://github.com/<usuario>/ticket-service]
Video de presentación	[COMPLETAR: enlace a la grabación de Teams]
Fecha	[COMPLETAR]

Índice

(Si el índice aparece vacío, haga clic derecho sobre él en Word → "Actualizar campos".)

1. Introducción

El presente documento describe el diseño, la implementación y el despliegue de un microservicio Cloud Native para una startup que requiere un **Sistema de Gestión de Reservas de Eventos con Tickets**. La solución genera tickets personalizados en formato PDF, los almacena temporalmente en **Amazon Elastic File System (EFS)**, los sube automáticamente a **Amazon S3** organizados por evento y usuario, y expone una API REST para generarlos, subirlos, descargarlos, modificarlos, eliminarlos, consultar su historial y reportar estadísticas de ventas y reservas.

El ciclo de entrega es completamente automatizado: cada *push* a la rama principal dispara un pipeline de **GitHub Actions** que compila y prueba el código, construye la imagen **Docker**, la publica en **Docker Hub** y la despliega en una instancia **Amazon EC2**. De esta forma se integran los conocimientos de Desarrollo Backend (Spring Boot, JPA, REST) con prácticas de despliegue continuo propias del desarrollo Cloud Native.

1.1 Objetivos

- Aprovisionar y configurar los componentes cloud requeridos (EC2, EFS, S3, IAM y Security Groups).
- Implementar un microservicio Spring Boot que integre EFS como almacenamiento temporal y S3 como almacenamiento durable.
- Automatizar la construcción, publicación y despliegue mediante un pipeline CI/CD.
- Verificar el funcionamiento end-to-end mediante pruebas en Postman sobre la instancia desplegada.

2. Análisis del caso y requisitos

La Tabla 1 relaciona cada requisito del caso con la forma en que se implementó en la solución.

Requisito del caso	Implementación
EFS para almacenamiento temporal	EFS montado en EC2 en `/mnt/efs` (NFS v4.1 con TLS) y expuesto al contenedor como `/data/efs`. Los PDF se guardan en `tickets/{evento}/{usuario}/ticketNNN.pdf`. `EfsStorageService`.
Subida automática a S3 por evento y usuario	Al generar un ticket se sube automáticamente con key `{evento}/{usuario}/ticketNNN.pdf` (ej. `evento123/usuarioX/ticket001.pdf`), con metadatos de fecha, usuario, evento y versión. `S3StorageService`.
Endpoint: generar tickets	`POST /api/tickets`
Endpoint: subir tickets a S3	`POST /api/tickets/{id}/subir` y `POST /api/tickets/subir-pendientes`
Endpoint: descargar por usuario o evento	`GET /api/tickets/{id}/descargar` (PDF) y `GET /api/tickets/descargar?eventoCodigo=&usuarioId=` (ZIP)
Endpoint: modificar detalles	`PUT /api/tickets/{id}`: regenera el PDF (nueva versión) y sobrescribe/mueve el objeto en S3

Requisito del caso	Implementación
Endpoint: eliminar tickets específicos	`DELETE /api/tickets/{id}`: borra de S3, EFS y BD, conservando el historial
Endpoint: estadísticas por evento	`GET /api/estadisticas/eventos` y `/api/estadisticas/eventos/{codigo}`
Imagen Docker publicada en Docker Hub al hacer push a main	Job `docker` del workflow `.github/workflows/ci-cd.yml` (tags `latest` y SHA del commit)
Despliegue automático en EC2 con GitHub Actions	Job `deploy`: SSH a la instancia, `docker pull` y `docker run` con el volumen EFS, más verificación de salud
Historial de archivos (criterio 5 de la pauta)	`GET /api/historial` con filtros por evento, usuario, ticket, acción y rango de fechas

Tabla 1. Trazabilidad requisito → implementación

3. Arquitectura de la solución

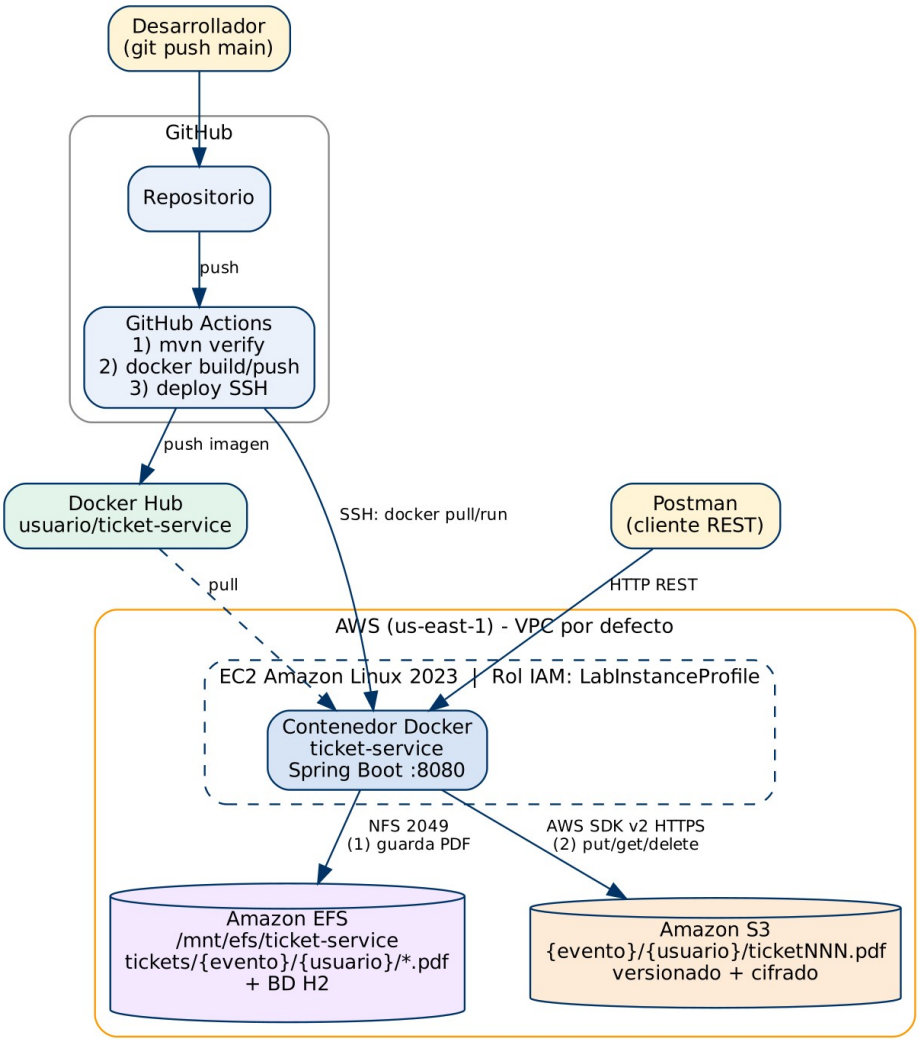


Figura 1. Arquitectura de despliegue y flujo CI/CD

La arquitectura sigue los principios de una aplicación **Twelve-Factor**: la configuración (bucket, región, ruta del EFS) se inyecta por variables de entorno, el proceso es *stateless* respecto del contenedor (todo el estado vive en EFS y S3) y el artefacto inmutable es la imagen Docker versionada por SHA de commit.

3.1 ¿Por qué EFS + S3?

- **EFS (almacenamiento de archivos compartido, NFS):** se monta como un directorio POSIX, por lo que la aplicación escribe el PDF con APIs estándar de Java (`java.nio.file`). A diferencia del disco efímero del contenedor, sobrevive a cada redeploy y podría compartirse entre varias instancias tras un balanceador. Se usa como zona de trabajo temporal (generación y regeneración del PDF). Además, aquí se guarda la base de datos H2 en modo archivo, de modo que los metadatos y el historial persisten entre despliegues.
- **S3 (almacenamiento de objetos):** es el repositorio durable y de bajo costo para los tickets finales. La organización por prefijos `{evento}/{usuario}/` permite listar y descargar por evento o usuario de forma eficiente. Se habilitó **versionado**, por lo que cada modificación conserva la versión anterior del ticket, y **cifrado SSE-S3**.
- Opcionalmente, con `EFS_DELETE_AFTER_UPLOAD=true` el PDF se elimina del EFS tras subirlo a S3, haciendo el almacenamiento en EFS estrictamente temporal; si luego se requiere re-subir, el servicio lo regenera.

3.2 Flujo de un ticket

1. El cliente invoca `POST /api/tickets`. Se valida el evento y su aforo disponible.
2. Se persiste el ticket (código `ticketNNN`) y se genera el PDF con Apache PDFBox.
3. El PDF se escribe en EFS de forma atómica (archivo `.tmp` + `move`) y se calcula su hash **SHA-256**.
4. Se sube a S3 enviando el checksum SHA-256 (S3 valida la integridad en el servidor) y metadatos.
5. Cada operación queda registrada en la tabla `historial_archivos` (GENERADO_EFS, SUBIDO_S3, MODIFICADO, DESCARGADO, ELIMINADO).

4. Aprovisionamiento de componentes cloud

El aprovisionamiento se realizó en la región **us-east-1** sobre la VPC por defecto. Los pasos se describen para la consola de AWS; el repositorio incluye además el script `deploy/aws-provision.sh` con los comandos equivalentes de AWS CLI (ejecutables en CloudShell).

4.1 Bucket S3

1. S3 → Create bucket. Nombre único (ej. `tickets-eventos-<sufijo>`), región us-east-1.
2. Mantener **Block all public access** activado: los tickets solo se entregan a través de la API.
3. Habilitar **Bucket Versioning** y cifrado por defecto **SSE-S3**.

[INSERTAR CAPTURA 1]
Bucket S3 creado con versionado y bloqueo de acceso público.

Figura 2. Bucket S3 creado con versionado y bloqueo de acceso público.

4.2 Security Groups

Security Group	Regla de entrada	Justificación
ticket-service-sg (EC2)	TCP 22 desde 0.0.0.0/0	SSH para el despliegue desde los runners de GitHub (IPs dinámicas). En producción se restringiría con un runner propio o SSM.
ticket-service-sg (EC2)	TCP 8080 desde 0.0.0.0/0	Acceso a la API REST para las pruebas con Postman.
ticket-service-efs-sg (EFS)	TCP 2049 (NFS) solo desde ticket-service-sg	Mínimo privilegio: solo la instancia de la aplicación puede montar el EFS.

4.3 Amazon EFS

1. EFS → Create file system → Customize. Nombre `ticket-service-efs`, cifrado en reposo activado, modo de rendimiento General Purpose, Throughput Bursting.
2. En **Network**, en la subred/zona de la instancia, asignar el Security Group `ticket-service-efs-sg` al *mount target*.
3. Copiar el **File system ID** (`fs-xxxxxxxx`) para el montaje.

[INSERTAR CAPTURA 2]
Sistema de archivos EFS con su mount target y Security Group.

Figura 3. Sistema de archivos EFS con su mount target y Security Group.

4.4 Instancia EC2 y rol IAM

1. EC2 → Launch instance: Amazon Linux 2023, tipo t2.micro, par de llaves `.pem`, Security Group `ticket-service-sg`.
2. En **Advanced details → IAM instance profile** seleccionar `LabInstanceProfile` (AWS Academy). Así el AWS SDK obtiene credenciales temporales del servicio de metadatos y no se almacenan llaves en el código ni en la imagen.
3. Mantener IMDSv2 obligatorio con *hop limit* 2 (necesario para que el contenedor alcance el servicio de metadatos).
4. Conectarse por SSH, copiar `deploy/setup-ec2.sh` y ejecutarlo con el ID del EFS. El script instala Docker y `amazon-efs-utils`, monta el EFS con TLS y lo registra en `/etc/fstab`:

```
sudo dnf install -y docker amazon-efs-utils
sudo systemctl enable --now docker && sudo usermod -aG docker ec2-user
sudo mkdir -p /mnt/efs
sudo mount -t efs -o tls fs-0123456789abcdef0:/ /mnt/efs
echo "fs-0123456789abcdef0:/ /mnt/efs efs _netdev,tls 0 0" | sudo tee -a /etc/fstab
sudo mkdir -p /mnt/efs/ticket-service && sudo chown 1000:1000 /mnt/efs/ticket-service
df -hT /mnt/efs        # Tipo: nfs4  -> EFS montado correctamente
```

[INSERTAR CAPTURA 3]
Salida de `df -hT /mnt/efs` en la EC2 mostrando el EFS montado (tipo nfs4).

Figura 4. Salida de `df -hT /mnt/efs` en la EC2 mostrando el EFS montado (tipo nfs4).

El directorio `/mnt/efs/ticket-service` se asigna al UID 1000 porque el contenedor se ejecuta con un usuario sin privilegios con ese UID (buena práctica de seguridad: no ejecutar procesos como root dentro del contenedor).

5. Desarrollo del microservicio

5.1 Tecnologías

Componente	Tecnología	Uso
Framework	Spring Boot 3.3.5 (Java 17)	Web, Data JPA, Validation, Actuator
Persistencia	H2 en modo archivo sobre EFS	Metadatos de eventos, tickets e historial
Almacenamiento de objetos	AWS SDK for Java v2 (S3)	put/get/head/delete/list de objetos
Generación de PDF	Apache PDFBox 3.0.3	Ticket personalizado con datos del evento y asistente
Pruebas	JUnit 5, MockMvc, Mockito	Ciclo completo con S3 simulado y EFS en directorio temporal
Contenedor	Docker multi-stage (temurin 17)	Imagen de ejecución liviana con usuario no root

5.2 Estructura del proyecto

```
ticket-service/
├── .github/workflows/ci-cd.yml        Pipeline CI/CD
├── Dockerfile  ·  docker-compose.local.yml
├── deploy/  setup-ec2.sh · aws-provision.sh · localstack-init.sh
├── postman/ticket-service.postman_collection.json
└── src/main/java/cl/duoc/tickets/
    ├── config/      AppProperties · S3Config · DataSeeder
    ├── controller/  Ticket · Evento · Historial · Estadisticas · Almacenamiento
    ├── dto/         Requests / Responses (records con Bean Validation)
    ├── exception/   GlobalExceptionHandler y excepciones de dominio
```

```
    ├── model/       Evento · Ticket · HistorialArchivo · enums
    ├── repository/  Spring Data JPA (+ Specifications para filtros)
    └── service/     TicketService · EfsStorageService · S3StorageService
                     TicketPdfGenerator · HistorialService · EstadisticasService
```

5.3 Almacenamiento temporal en EFS

`EfsStorageService` trata el EFS como un directorio local. La ruta raíz se configura con `EFS_BASE_PATH` y cada ticket se guarda en `tickets/{evento}/{usuario}/{ticket}.pdf`. La escritura es atómica para evitar que un ticket quede a medio escribir si el proceso se interrumpe, y toda ruta se normaliza y valida para que nunca salga del directorio raíz (prevención de *Path Traversal*, CWE-22):

```
public Path guardar(String evento, String usuario, String ticket, byte[] contenido) {
    Path destino = resolver(evento, usuario, ticket);          // valida segmentos y raiz
    Files.createDirectories(destino.getParent());
    Path tmp = destino.resolveSibling(destino.getFileName() + ".tmp");
    Files.write(tmp, contenido);
    Files.move(tmp, destino, REPLACE_EXISTING, ATOMIC_MOVE);  // escritura atomica en NFS
    return destino;
}
```

El endpoint `GET /api/almacenamiento/efs/estado` informa el tipo de sistema de archivos (`nfs4` cuando el EFS está montado), si es escribible y el espacio disponible, lo que permite evidenciar en la demostración que la aplicación realmente escribe en EFS.

5.4 Subida y organización en S3

La key se construye con `RutaArchivoUtil.rutaRelativa(evento, usuario, ticket)`, que aplica una lista blanca `^[A-Za-z0-9_-]{1,60}$` a cada segmento. La subida envía el checksum SHA-256 para que S3 rechace el objeto si se corrompió en tránsito, y agrega metadatos que permiten auditar el objeto desde la consola:

```
PutObjectRequest req = PutObjectRequest.builder()
        .bucket(bucket).key("evento123/usuarioX/ticket001.pdf")
        .contentType("application/pdf")
        .checksumAlgorithm(ChecksumAlgorithm.SHA256)
        .metadata(Map.of("evento", "evento123", "usuario", "usuarioX",
                         "version", "1", "fecha-generacion", "2026-11-09T21:10:00"))
        .build();
s3.putObject(req, RequestBody.fromBytes(pdf));
```

5.5 Modificación y actualización en S3

`PUT /api/tickets/{id}` aplica solo los campos enviados, incrementa la **versión** del ticket, regenera el PDF en EFS y **sobrescribe** el objeto en S3 con la misma key (el versionado del bucket conserva la versión previa). Si cambia el `usuarioId` (transferencia del ticket), el archivo se **mueve**: se sube a la nueva carpeta `{evento}/{nuevoUsuario}/` y se elimina la key anterior, manteniendo la organización por usuario coherente. Si un ticket cancelado se reactiva, se vuelve a validar el aforo del evento.

5.6 Descarga con verificación de integridad

Las descargas se hacen siempre desde S3 (fuente de verdad). Antes de responder, el servicio recalcula el SHA-256 del contenido y lo compara con el registrado al generar el ticket; si no coincide responde un error en lugar de entregar un archivo corrupto. El hash se devuelve en la cabecera `X-Checksum-SHA256`. La descarga por usuario o evento entrega un ZIP que conserva la estructura `evento/usuario/ticket.pdf`.

5.7 Historial de archivos

Cada operación sobre archivos genera un registro inmutable en `historial_archivos` con ticket, evento, usuario, acción, ubicación (ruta EFS o URI `s3://`), versión, checksum y fecha. El registro no tiene clave foránea al ticket, por lo que se conserva aun después de eliminarlo. La consulta `GET /api/historial` acepta filtros combinables (`eventoCodigo`, `usuarioId`, `ticketCodigo`, `accion`, `desde`, `hasta`) construidos con **JPA Criteria API**, lo que evita concatenar SQL (prevención de inyección, OWASP A03).

5.8 Endpoints REST

Método	Ruta	Descripción
POST	/api/tickets?subirS3=	Genera ticket en EFS y lo sube a S3 (por defecto)
POST	/api/tickets/{id}/subir	Sube o re-sube un ticket a S3
POST	/api/tickets/subir-pendientes	Sube todos los tickets que aún no están en S3
GET	/api/tickets/{id}/descargar	Descarga PDF desde S3 (verifica SHA-256)
GET	/api/tickets/descargar? eventoCodigo=&usuarioId=	Descarga ZIP por evento y/o usuario
PUT	/api/tickets/{id}	Modifica detalles; regenera y actualiza en S3
DELETE	/api/tickets/{id}	Elimina de S3, EFS y BD
GET	/api/tickets? eventoCodigo=&usuarioId=&estado=	Lista tickets con filtros
GET	/api/historial?...	Historial de archivos con filtros
GET	/api/estadisticas/eventos[/{codigo}]	Ventas, reservas, recaudación, ocupación
GET	/api/almacenamiento/efs/estado | efs/archivos | s3/archivos	Inspección de EFS y S3
GET/ POST	/api/eventos	Listar / crear eventos

5.9 Estadísticas de ventas y reservas

Por cada evento se informa: total de tickets, vendidos, reservados, cancelados, disponibles respecto del aforo, porcentaje de ocupación, recaudación (suma de tickets VENDIDO), monto en reservas, distribución por categoría, usuarios distintos y cantidad de archivos en S3.

6. Pipeline CI/CD

El workflow `.github/workflows/ci-cd.yml` se ejecuta con cada `push` a `main` (y manualmente con `workflow_dispatch`). Está dividido en tres jobs encadenados con `needs`, de modo que una falla en las pruebas impide publicar o desplegar una imagen defectuosa (*quality gate*):

Job	Pasos	Resultado
build-test	checkout → setup-java 17 (caché Maven) → `mvn -B clean verify` → publica reportes de pruebas	Código compilado y pruebas aprobadas
docker	login Docker Hub (token) → Buildx → build multi-stage → push tags `latest` y `<sha>`	Imagen versionada en Docker Hub
deploy	SSH a EC2 (appleboy/ssh-action) → verifica montaje EFS → `docker pull` → reemplaza contenedor con `-v /mnt/efs/ticket-service:/data/efs` → health check `/actuator/health`	Nueva versión en ejecución en EC2

Fragmento del job de despliegue:

```
docker pull "$IMAGE"
docker rm -f ticket-service 2>/dev/null || true
docker run -d --name ticket-service --restart unless-stopped \
  -p 8080:8080 \
  -v /mnt/efs/ticket-service:/data/efs \
  -e EFS_BASE_PATH=/data/efs -e S3_BUCKET="$S3_BUCKET" -e AWS_REGION="$AWS_REGION" \
  "$IMAGE"
for i in $(seq 1 30); do
  curl -fs http://localhost:8080/actuator/health | grep -q '"UP"' && exit 0; sleep 4
done; docker logs --tail 100 ticket-service; exit 1
```

6.1 Secrets del repositorio

Secret	Contenido
DOCKERHUB_USERNAME / DOCKERHUB_TOKEN	Usuario y *access token* de Docker Hub (no la contraseña)
EC2_HOST / EC2_USER	IP pública de la instancia / `ec2-user`
EC2_SSH_KEY	Contenido del archivo `.pem` de la instancia
S3_BUCKET / AWS_REGION	Nombre del bucket / `us-east-1`
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY / AWS_SESSION_TOKEN	Opcionales: solo si la EC2 no tiene el rol `LabInstanceProfile`. En AWS Academy expiran con cada sesión del laboratorio.

[INSERTAR CAPTURA 4]
Ejecución exitosa del workflow en la pestaña Actions (3 jobs en verde).

Figura 5. Ejecución exitosa del workflow en la pestaña Actions (3 jobs en verde).

[INSERTAR CAPTURA 5]
Repositorio en Docker Hub con los tags latest y SHA del commit.

Figura 6. Repositorio en Docker Hub con los tags latest y SHA del commit.

[INSERTAR CAPTURA 6]
`docker ps` en la EC2 mostrando el contenedor ticket-service en ejecución.

Figura 7. `docker ps` en la EC2 mostrando el contenedor ticket-service en ejecución.

7. Pruebas con Postman

Se entrega la colección `postman/ticket-service.postman_collection.json` con 36 solicitudes organizadas en 9 carpetas, en el orden de la demostración. Cada solicitud incluye *tests* automáticos (código de estado, estructura de la key S3, cabecera de checksum, etc.) y variables encadenadas (`ticketId`, `ticketId2`). Basta con cambiar `baseUrl` a `http://<IP_PUBLICA_EC2>:8080`.

Carpeta	Qué demuestra	Criterio pauta
0. Verificación	Health, eventos precargados, estado del EFS (nfs4)	1
1. Generar tickets	PDF en EFS + subida automática; ticket solo en EFS	1, 2
2. Subir a S3	Subida manual, pendientes y listado por prefijo evento/usuario	2
3. Descargar	PDF individual con checksum; ZIP por usuario, evento y ambos	4
4. Modificar	Cambio de datos (nueva versión), confirmación de reserva, transferencia de usuario	3
5. Historial	Filtros por evento, usuario, acción, fechas y combinados	5
6. Estadísticas	Ventas y reservas por evento	Requisito del caso
7. Eliminar	Borrado en S3/EFS/BD y registro ELIMINADO en historial	Requisito del caso
8. Validaciones	Path traversal (400), evento inexistente (404), aforo agotado (409)	Calidad / seguridad

[INSERTAR CAPTURA 7]
POST /api/tickets: respuesta 201 con rutaEfs y s3Key evento123/usuarioX/ticketNNN.pdf.

Figura 8. POST /api/tickets: respuesta 201 con rutaEfs y s3Key evento123/usuarioX/ticketNNN.pdf.

[INSERTAR CAPTURA 8]
Consola de S3 mostrando la estructura de carpetas evento123/usuarioX/.

Figura 9. Consola de S3 mostrando la estructura de carpetas evento123/usuarioX/.

[INSERTAR CAPTURA 9]
PUT /api/tickets/{id}: versión 2 y objeto actualizado en S3 (pestaña Versions).

Figura 10. PUT /api/tickets/{id}: versión 2 y objeto actualizado en S3 (pestaña Versions).

[INSERTAR CAPTURA 10]
GET /api/tickets/descargar?usuarioId=usuarioX: ZIP descargado.

Figura 11. GET /api/tickets/descargar?usuarioId=usuarioX: ZIP descargado.

[INSERTAR CAPTURA 11]
GET /api/historial con filtros eventoCodigo y accion.

Figura 12. GET /api/historial con filtros eventoCodigo y accion.

[INSERTAR CAPTURA 12]
GET /api/estadisticas/eventos/evento123.

Figura 13. GET /api/estadisticas/eventos/evento123.

8. Seguridad y calidad aplicadas

Práctica	Implementación	Riesgo mitigado
Sin credenciales en el código	Rol IAM de instancia + cadena de credenciales del SDK; secrets de GitHub	Fuga de llaves (OWASP A07/A05)
Validación de entradas	Bean Validation en DTOs y lista blanca en segmentos de ruta	Path Traversal (CWE-22), keys S3 maliciosas
Consultas parametrizadas	Spring Data JPA + Criteria API para filtros dinámicos	Inyección SQL (OWASP A03)

Práctica	Implementación	Riesgo mitigado
Integridad de archivos	SHA-256 al generar, validado por S3 al subir y por la app al descargar	Archivos corruptos o alterados
Mínimo privilegio en red	NFS 2049 solo desde el SG de la EC2; bucket sin acceso público	Acceso no autorizado a datos
Cifrado	EFS cifrado en reposo y montado con TLS; S3 con SSE-S3 y acceso HTTPS	Exposición de datos (OWASP A02)
Contenedor sin root	Usuario UID 1000 en la imagen	Escalamiento de privilegios
Manejo de errores	`GlobalExceptionHandler` con respuestas JSON uniformes sin stacktrace	Divulgación de información
Quality gate en CI	`mvn verify` con pruebas de integración antes de construir la imagen	Desplegar versiones defectuosas

En cuanto a calidad (ISO/IEC 25010), la solución apunta a la **fiabilidad** (health check y reinicio automático `--restart unless-stopped`, datos en EFS que sobreviven a redeploys), la **mantenibilidad** (capas separadas controlador-servicio-repositorio, configuración externalizada) y la **portabilidad** (imagen Docker y `docker-compose.local.yml` con LocalStack para ejecutar sin cuenta AWS).

8.1 Pruebas automatizadas

`TicketLifecycleIntegrationTest` levanta el contexto completo de Spring con S3 simulado por Mockito y el EFS en un directorio temporal, y recorre el ciclo generar → descargar → modificar (con cambio de usuario) → ZIP → historial → estadísticas → eliminar. Además, verifica el rechazo de *path traversal* y el control de aforo. `RutaArchivoUtilTest` cubre la construcción de keys y prefijos.

9. Correspondencia con la pauta de evaluación

#	Criterio (puntaje)	Evidencia en la solución
1	Configura el almacenamiento temporal en EFS (15)	Secciones 4.3, 4.4 y 5.3; `/api/almacenamiento/efs/estado` y `efs/archivos`; `df -hT`
2	Sube los archivos generados a AWS S3 (10)	Secciones 3.2 y 5.4; key `{evento}/{usuario}/ticketNNN.pdf` con metadatos de fecha; `s3/archivos`
3	Modifica y actualiza archivos en S3 (15)	Sección 5.5; PUT con versión, sobrescritura y movimiento de key; versionado del bucket
4	Descarga archivos desde S3 (10)	Sección 5.6; PDF individual y ZIP por usuario/evento con verificación SHA-256
5	Consulta el historial de archivos generados (10)	Sección 5.7; `/api/historial` con 6 filtros combinables
6	Despliega mediante pipeline automatizado (20)	Sección 6; workflow con build/test, Docker Hub y deploy en EC2

#	Criterio (puntaje)	Evidencia en la solución
7	Presenta la solución en un video (20)	Guion de video incluido (`Guion_video.md`) siguiendo el orden de la colección Postman

10. Conclusiones

Se construyó y desplegó un microservicio Cloud Native que resuelve el caso de la startup combinando dos servicios de almacenamiento con propósitos complementarios: EFS como sistema de archivos compartido y persistente para la generación de tickets, y S3 como repositorio durable, versionado y organizado por evento y usuario. El uso de prefijos jerárquicos en S3 simplificó tanto la descarga por usuario o evento como la auditoría desde la consola, y el historial de operaciones entrega trazabilidad completa del ciclo de vida de cada archivo.

El pipeline de GitHub Actions convierte cada *push* a `main` en una entrega verificable: las pruebas actúan como compuerta de calidad, la imagen se versiona por commit en Docker Hub y el despliegue en EC2 se valida con un health check, eliminando pasos manuales propensos a error. Como mejoras futuras se proponen: reemplazar H2 por Amazon RDS para escalar a múltiples instancias, publicar la API detrás de un Application Load Balancer con HTTPS, usar URLs prefirmadas de S3 para descargas directas, autenticar la API con JWT y reemplazar el acceso SSH del pipeline por AWS Systems Manager.

Anexo: enlaces de entrega

Entregable	Enlace / archivo
Repositorio GitHub	[COMPLETAR]
Imagen Docker Hub	[COMPLETAR: https://hub.docker.com/r/<usuario>/ticket-service]
Video de presentación (Teams)	[COMPLETAR]
Colección Postman	`postman/ticket-service.postman_collection.json`